



西安交通大学
XI'AN JIAOTONG UNIVERSITY

机器学习

实验报告

基于 BERT-base-chinese 微调的中文上下文 问答模型

2024-2025 学年 第二学期

学 院： 数学与统计学院
班 级： 数学强基 2202
学 号： 2224110096
姓 名： 陈宇昂
指导教师： 孟德宇老师
实验地点： 数学实验中心
完成时间： 2025-04-03

目录

1	实验内容	1
1.1	问题描述	1
1.2	实验步骤	1
1.3	实验要求	1
2	实验描述	2
2.1	数据集与实验环境参数	2
2.2	算法描述	2
2.2.1	BERT 模型架构	3
2.2.2	预训练机制	5
2.2.3	微调策略	6
2.3	模块化设计	6
2.3.1	数据集模块	6
2.3.2	模型的建立	7
2.3.3	可视化	8
2.3.4	模型的训练和评估	8
2.4	程序框图	9
3	实验结果与所遇问题	10
3.1	实验结果分析	10
3.2	问题与解决措施	12
A	附录：实验代码	14
A.1	QA_model.py	14
A.2	QA_test.py	20
A.3	QA_Interaction.py	24

1 实验内容

1.1 问题描述

本实验旨在基于 BERT-base-chinese 模型,在小样本数据集(训练集 221 篇文章,验证集、测试集各 24 篇文章,问题若干)与低算力设备(NVIDIA GeForce RTX 3050 Ti)的限制下,通过微调方法构建中文上下文问答模型。核心目标为:

1. 解决长文本输入限制,通过滑动窗口策略保留关键语义信息;
2. 探索各超参数对于模型性能的影响。

1.2 实验步骤

1. **数据预处理:** 解析 JSON 格式数据,过滤答案起止位置超界的样本,提取上下文、问题及答案起止索引,构建标准化问答对。
2. **模型架构设计:** 加载预训练 BERT-base-chinese 模型,调整输出层为问答任务适配的起始/终止位置分类器。
3. **滑动窗口实现:** 设置最大序列长度 512,窗口步长 128,通过分块编码处理超长文本,维护上下文连贯性。
4. **训练流程优化:** 采用 AdamW 优化器(学习率 $3e-5$),批次大小 4,单 GPU 单周期训练,监控训练与验证损失曲线。
5. **模型性能验证:** 测试集上计算答案匹配准确率,并通过 QA_Interaction.py 进行交互式问答验证实际推理效果。
6. **结果可视化:** 绘制损失变化曲线,分析模型收敛趋势与过拟合风险。

1.3 实验要求

1. 在合理训练时间内完成模型微调,使得模型基本具有根据上下文回答问题的能力;
2. 测试集准确率需达到基线模型(未微调 BERT)的 1.5 倍以上。

2 实验描述

2.1 数据集与实验环境参数

数据集构建

1. **数据来源**: 原始数据取自[苏剑林](#)的网站“[科学空间](#)” [1] 中已经过处理的中文 SQuAD 格式数据集, 包含 train-v1.1-zh.json(29.7MB, 442 篇文章, 651,292 行) 与 dev-v1.1-zh.json(5.38MB, 48 篇文章, 143,306 行) 两个文件。
2. **数据分割**:
 - (a) 训练集 (train.json): 来自于 train-v1.1-zh.json 前 221 篇文章 (15.2MB, 651,292 行);
 - (b) 验证集 (dev.json): 来自于 dev-v1.1-zh.json 的前 24 篇文章 (2.83MB, 77,399 行);
 - (c) 测试集 (test.json): 来自于 dev-v1.1-zh.json 的后 24 篇文章 (2.54MB, 65,911 行)。

实验环境参数 本次实验的相关参数如表1所示。

表 1: 实验运行相关参数

项目	参数
执行平台	PyCharm
主要依赖库	Pytorch, Transformers
基线模型	bert-base-chinese
Python 版本	Python 3.11.8
GPU 型号	NVIDIA GeForce RTX 3050 Ti
CUDA 版本	CUDA 12.7

2.2 算法描述

本实验基于 BERT 框架 [2],调用了[HuggingFace](#)中的 BERT-base-chinese 作为基线模型, 针对中文问答场景进行微调优化。以下我将从 **BERT** 模型架构、预训练机制、微调策略这三点对本实验的算法思路进行说明。

2.2.1 BERT 模型架构

输入表示 输入序列由问题 Q 与上下文 C 拼接而成，形式化为：

$$X = [[CLS]; Q; [SEP]; C; [SEP]]$$

其中特殊标记 $[CLS]$ 表示分类信号， $[SEP]$ 用于分隔不同文本段。输入向量通过三重嵌入的逐元素和构成：

$$E_i = E_{\text{token}}^{(i)} + E_{\text{position}}^{(i)} + E_{\text{segment}}^{(i)}$$

具体分解如下：

1. **词嵌入**：采用 WordPiece 分词器将文本切分为子词单元，映射为 768 维向量。对于中文，子词单元覆盖常见汉字与高频词组。
2. **位置嵌入**：编码绝对位置信息，通过正弦 - 余弦函数生成：

$$PE_{(pos, 2k)} = \sin\left(\frac{pos}{10000^{2k/d}}\right), \quad PE_{(pos, 2k+1)} = \cos\left(\frac{pos}{10000^{2k/d}}\right)$$

其中 $pos \in [0, 511]$ 为位置索引， $d = 768$ 为嵌入维度， k 为维度索引。该设计使得模型能通过线性变换捕捉相对位置关系。

3. **段嵌入**：区分问题与上下文，设问题段为 $e_0 \in \mathbb{R}^{768}$ ，上下文段为 $e_1 \in \mathbb{R}^{768}$ 。对于单段输入（如分类任务），仅使用 e_0 。

输入构造示例：

1 [CLS] 什么公司拥有美国广播公司？ [SEP] 美国广播公司（ABC）... [SEP]

其中， $[CLS]$ 符号的最终隐藏状态用于分类任务， $[SEP]$ 符号分割不同段落。

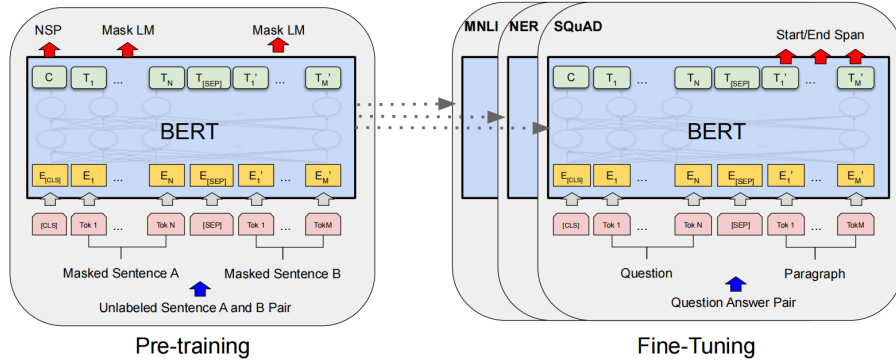


图 1: 微调 BERT 在中文问答模型中的示意图

Transformer 编码器 BERT 的核心由多层 Transformer 编码器堆叠而成。与传统单向语言模型不同，其创新之处在于**双向自注意力机制**。每个编码层通过多头注意力（Multi-Head Attention）捕捉全局依赖关系：

1. **多头自注意力**：将输入矩阵 $X \in \mathbb{R}^{n \times d}$ 投影至 $h = 12$ 个独立子空间：

$$Q^{(i)} = XW_Q^{(i)}, \quad K^{(i)} = XW_K^{(i)}, \quad V^{(i)} = XW_V^{(i)}, \quad i = 1, \dots, h$$

其中 $W_Q^{(i)}, W_K^{(i)}, W_V^{(i)} \in \mathbb{R}^{d \times d/h}$ 为可学习参数， $d/h = 64$ 。每个头计算缩放点积注意力：

$$\text{Attention}^{(i)} = \text{softmax} \left(\frac{Q^{(i)} K^{(i)\top}}{\sqrt{d/h}} \right) V^{(i)}$$

各头输出拼接后经线性变换：

$$\text{MHA}(X) = \text{Concat}(\text{Attention}^{(1)}, \dots, \text{Attention}^{(h)})W_O$$

其中 $W_O \in \mathbb{R}^{d \times d}$ 为输出权重矩阵。

2. **前馈神经网络**：对自注意力输出进行非线性变换：

$$\text{FFN}(X) = \text{GELU}(XW_1 + b_1)W_2 + b_2$$

其中 $W_1 \in \mathbb{R}^{768 \times 3072}$, $W_2 \in \mathbb{R}^{3072 \times 768}$ ，GELU 激活函数定义为：

$$\text{GELU}(x) = x\Phi(x) \approx 0.5x \left(1 + \tanh \left(\sqrt{2/\pi}(x + 0.044715x^3) \right) \right)$$

3. **残差连接与层归一化**：每个子模块输出与输入相加后归一化：

$$X' = \text{LayerNorm}(X + \text{MHA}(X)), \quad X'' = \text{LayerNorm}(X' + \text{FFN}(X'))$$

输出表示 最终编码层输出 $H^{(12)} \in \mathbb{R}^{n \times 768}$ 包含两类关键信息：

1. '[CLS]' 标记向量： $h_{[\text{CLS}]} \in \mathbb{R}^{768}$ ，用于分类任务；
2. 上下文向量序列：第 i 个 token 的隐状态 $h_i \in \mathbb{R}^{768}$ ，在问答任务中用于预测答案跨度。

表 2: 模型性能对比

对比维度	BERT	预训练单向模型（GPT）
上下文建模	全双向注意力	单向历史依赖
长文本处理	滑动窗口 + 偏移映射	分层处理

2.2.2 预训练机制

BERT 的预训练通过无监督任务学习通用语言表示，其核心在于掩码语言模型（Masked Language Model, MLM）与下一句预测（Next Sentence Prediction, NSP）的联合优化。

掩码语言模型 核心思想：通过随机掩盖部分 token 并预测其原始值，强制模型学习上下文双向依赖。

随机遮蔽输入序列中 15% 的 token，目标为基于双向上下文预测被遮蔽词。具体遮蔽策略为：

$$\text{Mask}(w_i) = \begin{cases} [\text{MASK}] & \text{以概率 0.8 替换} \\ \text{随机词} \sim \mathcal{V} & \text{以概率 0.1 替换} \\ w_i & \text{以概率 0.1 保留} \end{cases}$$

其中 $\mathcal{V}$ 为词表。MLM 损失函数为交叉熵：

$$\mathcal{L}_{\text{MLM}} = - \sum_{i \in \mathcal{M}} \log P(w_i | X_{\setminus \mathcal{M}})$$

其中 $\mathcal{M}$ 为被遮蔽位置集合， $X_{\setminus \mathcal{M}}$ 表示未遮蔽的上下文。概率计算通过 Softmax 归一化：

$$P(w_i | X_{\setminus \mathcal{M}}) = \frac{\exp(e_{w_i}^\top h_i)}{\sum_{v \in \mathcal{V}} \exp(e_v^\top h_i)}$$

其中 e_v 为词 v 的嵌入向量， h_i 为位置 i 的隐状态。

下一句预测 目标：提升模型对句子间逻辑关系的理解能力。

给定句子对 (A, B) ，判断 B 是否为 A 的实际后续句子。正样本从文档中连续抽取，负样本随机替换 B 。NSP 损失函数为二元交叉熵：

$$\mathcal{L}_{\text{NSP}} = -y \log p(\text{IsNext}) - (1 - y) \log(1 - p(\text{IsNext}))$$

其中 $y \in \{0, 1\}$ 为标签，概率 $p(\text{IsNext})$ 通过 [CLS] 向量计算：

$$p(\text{IsNext}) = \sigma(w_{\text{nsf}}^\top h_{[\text{CLS}]} + b_{\text{nsf}})$$

σ 为 Sigmoid 函数， $w_{\text{nsf}} \in \mathbb{R}^{768}$ 为可学习参数。

2.2.3 微调策略

针对小样本问答任务与低算力约束，本实验使用多种策略以平衡性能与效率。

动态滑动窗口 为处理长文本输入，将上下文分割为重叠窗口（长度 $L = 150$ ，步长 $S = 128$ ），各窗口与问题拼接为独立输入，

$$\text{Input}_k = [[\text{CLS}]; Q; [\text{SEP}]; C_k; [\text{SEP}]]$$

编码后对各窗口的答案概率加权融合，选择最大置信度区间作为最终答案。

轻量化输出层 在 BERT 顶层添加可训练参数 $S, E \in \mathbb{R}^{768}$ ，分别预测答案起止位置。对于第 i 个 token，其作为答案起点的概率为：

$$P_{\text{start}}(i) = \frac{\exp(S^\top h_i^{(12)})}{\sum_{j=1}^n \exp(S^\top h_j^{(12)})}$$

同理定义 $P_{\text{end}}(i)$ 。损失函数为起止位置的负对数似然，

$$\mathcal{L}_{\text{QA}} = -\log P_{\text{start}}(s^*) - \log P_{\text{end}}(e^*)$$

其中 (s^*, e^*) 为真实答案的起止位置。

2.3 模块化设计

本实验采用模块化设计理念，将问答系统拆分为数据集模块、模型构建、可视化、模型的训练和评估四大核心模块。

2.3.1 数据集模块

数据加载与预处理

1. 通过函数 `load_data(file_path)` 读取 JSON 格式的数据文件。该函数在读取数据时，采用 UTF-8 编码并对数据进行基本格式验证，以确保数据的完整性。
2. 函数 `preprocess_data(data)` 对原始数据进行逐条解析，将每个问题及其对应的上下文、答案文本、答案在上下文中的起始位置和结束位置等信息提取出来。

在上述过程中，数据加载器还对答案为空或答案起始位置错误的样本进行了过滤处理，从而保证了数据的有效性和容错性。

数据集类封装 为了便于数据批处理和多样本输入，本项目设计了数据集类 `SquadDataset`。

1. 利用 BERT 的 Fast Tokenizer (`BertTokenizerFast`) 进行快速分词和编码，调用内部函数 `encode_plus()` 生成标准的输入格式，包括输入 ID (`input_ids`)、注意力掩码 (`attention_mask`) 和字符偏移映射 (`offset_mapping`)；
2. 为了应对长文本的输入，本模块引入了滑动窗口策略（使用全局变量 `STRIDE`），将过长的段落切分为多个重叠分块，并为每个分块重新生成相应的标签位置；
3. `SquadDataset` 在 `__getitem__` 方法中返回一个列表，每个元素为一个字典，包含张量化后的输入、注意力掩码以及对应的答案起始和结束位置。

这样的设计既便于 `DataLoader` 进行后续批处理，又确保每个样本的多分块数据均被利用到模型训练中。

2.3.2 模型的建立

本项目采用 BERT-base-chinese 作为基础模型，因此在模型建立时主要依赖预训练模型进行微调，并通过自定义的训练函数进行参数更新。

基础模型加载与初始化 在文件 `QA_model.py` 中,通过调用 `BertForQuestionAnswering.from_pretrained('bert-base-chinese')` 加载预训练模型。为保证实验的可重复性，系统在加载模型前调用了 `same_seeds(seed)` 函数，对 PyTorch 随机种子进行设置，从而使模型在训练时具有确定性。

训练输入构造与分块处理 针对中文长文本问答任务，为适应模型最大输入长度的限制，本模块使用了函数 `padding(ids_q, ids_p)`：

1. 用于对问题和分块后的上下文进行拼接，并在两部分之间插入特殊标记（例如 `[CLS]` 和 `[SEP]`）
2. 对不足最大长度的部分进行零填充，同时生成对应的 `token type` 标记和注意力掩码

在数据预处理过程中，为确保答案位置与分块数据对齐，系统遍历每个分块的偏移映射，根据字符级位置确定对应的 `token` 索引，从而构造精确的标签数据。

模型接口设计 在构建完训练数据后,通过统一的接口函数(例如 `answer_question(context, question)`)封装了模型预测流程。该函数实现了对输入问题和上下文的编码、分块处理、模型预测以及结果后处理,其中利用函数 `evaluate()` 对各个分块输出的起始和结束位置进行比较,并最终选取得分最高的答案作为预测输出。

2.3.3 可视化

训练过程监控 在训练函数 `train()` 中,利用 `tqdm` 进度条实时显示模型的训练进度和平均损失情况。每一步的训练损失和验证损失均被记录在对应的列表中,以便后续绘图分析。

损失曲线绘制 训练完成后,通过 `Matplotlib` 绘制训练损失和验证损失随训练步数变化的曲线图。图中采用 `Seaborn` 的配色风格(例如:训练损失采用色值 `#70FFE2`,验证损失采用色值 `#FF6C6C`),并设置图例、网格和标签,直观展示训练过程中的收敛情况。图形通过 `plt.savefig()` 函数保存为 `PNG` 格式文件,便于后续实验报告的整理和归档。

2.3.4 模型的训练和评估

模型训练和评估模块是整个实验的核心,直接决定了模型的最终性能。在本项目中,训练和评估流程主要包含数据展开、批量训练、验证以及测试四个部分,具体设计如下。

训练流程 训练模块在 `train()` 函数中实现,

1. 加载预训练的 `BERT` 模型和对应的 `Fast Tokenizer`,并将模型部署在合适的设备上(`CUDA` 或 `CPU`)。
2. 加载数据后,通过数据集类 `SquadDataset` 构造原始数据列表,并对数据进行展平操作,将每个样本的多个分块数据整合成统一的训练集。然后,利用 `PyTorch` 的 `DataLoader` 对训练数据进行批处理,每个 `batch` 的数据包括输入 `ID`、注意力掩码以及答案起始和结束标签。
3. 训练模型,在每次迭代中计算损失,通过反向传播更新参数,并实时记录训练损失。在每一步训练中,优化器(`AdamW`)根据当前损失和梯度调整参数,从而逐步降低模型预测误差。

验证与评估

1. 在每个 epoch 结束后，调用 `model.eval()`，利用验证集进行模型性能的即时检测。验证过程中，系统同样对每个 batch 进行前向传播，并计算验证损失，确保模型在未见数据上的泛化能力。
2. 在测试脚本（`QA_test.py`）中，通过遍历整个测试集，对每个问题调用封装好的预测接口 `answer_question()`，并将模型输出的答案与标准答案进行比对，计算整体准确率。

结果展示与模型保存 训练结束后，模型和 Tokenizer 通过 `model.save_pretrained()` 和 `tokenizer.save_pretrained()` 函数分别保存到指定目录。与此同时，前文绘制的训练和验证损失曲线也会被存储为图片文件，便于结果展示和归档。

2.4 程序框图

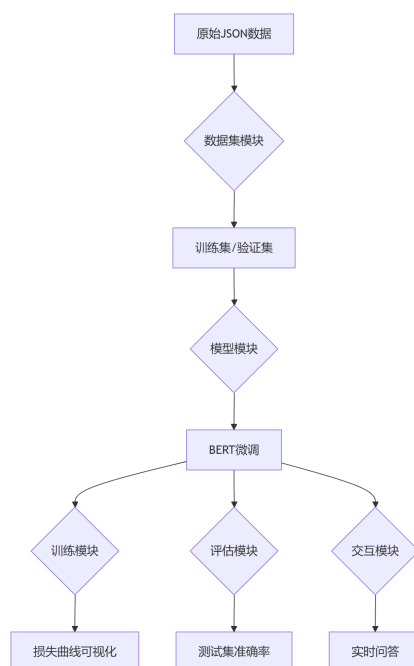


图 2: 程序框图

具体代码实现见附录。

3 实验结果与所遇问题

3.1 实验结果分析

在小样本数据集（训练集 221 篇文章，验证集、测试集各 24 篇文章，问题若干）的条件下，我分别尝试了多组超参数并在 `test.json` 上测试了模型的回答准确率，结果如下图所示。总体来看，基线模型 BERT-base-chinese 在有限训练步数下即可获得一定水平的正确率，但在输入长度、批大小及学习率等超参数方面的微调会对模型性能产生较大影响。

```
Processing questions: 100% | 3496/3496
使用模型: bert-base-chinese
总问题个数: 3496
回答正确问题个数: 2
正确率: 0.06%

进程已结束, 退出代码为 0
```

(a) 基线模型 BERT-base-chinese

```
Processing questions: 100% | 3496/3496
使用模型: bert_qa_model_256_3e-5_4_128
总问题个数: 3496
回答正确问题个数: 1371
正确率: 39.22%

进程已结束, 退出代码为 0
```

(b) 最大输入 = 256, 批大小 = 4, 滑动窗口 = 128, 学习率 = 3×10^{-5}

```
Processing questions: 100% | 3496/3496
使用模型: bert_qa_model_256_1e-5_4_128
总问题个数: 3496
回答正确问题个数: 1275
正确率: 36.47%

进程已结束, 退出代码为 0
```

(c) 最大输入 = 256, 批大小 = 4, 滑动窗口 = 128, 学习率 = 1×10^{-5}

```
Processing questions: 100% | 3496/3496
使用模型: bert_qa_model_256_3e-5_4_64
总问题个数: 3496
回答正确问题个数: 971
正确率: 27.77%

进程已结束, 退出代码为 0
```

(d) 最大输入 = 256, 批大小 = 4, 滑动窗口 = 64, 学习率 = 3×10^{-5}

```
Processing questions: 100% | 3496/3496
使用模型: bert_qa_model_256_3e-5_8_128
总问题个数: 3496
回答正确问题个数: 1251
正确率: 35.78%

进程已结束, 退出代码为 0
```

(e) 最大输入 = 256, 批大小 = 8, 滑动窗口 = 128, 学习率 = 3×10^{-5}

```
Processing questions: 100% | 3496/3496
使用模型: bert_qa_model_512_3e-5_4_128
总问题个数: 3496
回答正确问题个数: 1459
正确率: 41.73%

进程已结束, 退出代码为 0
```

(f) 最大输入 = 512, 批大小 = 4, 滑动窗口 = 128, 学习率 = 3×10^{-5}

图 3: 各参数组合训练所得模型在 `test.json` 数据集下的正确率

从图3a可见，使用基线模型 BERT-base-chinese 直接在 `test.json` 上进行测试，得到的结果正确率极低。

图3b3c3e展示了最大输入长度为 256 时，在不同批大小与学习率设定下的测试结果：（图3b3e）当批大小从 4 增至 8、学习率固定为 3×10^{-5} ，批量的增大会导致模型的泛化能力下降，导致测试准确率下降；（图3b3c）当批大小固定时，应使用适宜大小的学习率，较低的学习率可能会陷入局部最优导致模型性能不佳。类似地，比较图3d3e3f，可以看出，增大输入长度和滑动窗口长度有助于模型捕捉更丰富的上下文信息，但也会带来训练时间的上升。

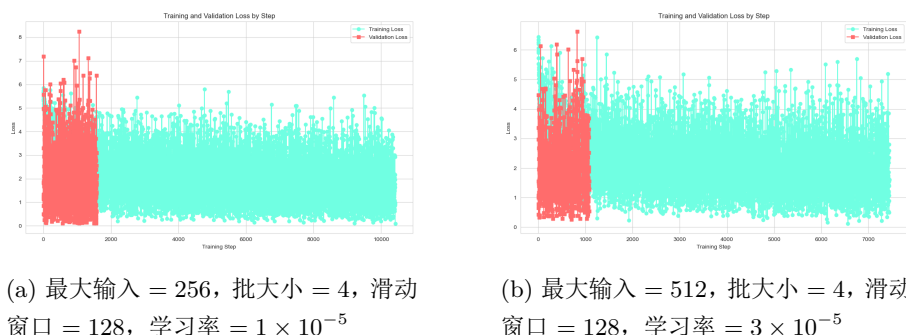


图 4: 各参数下损失曲线

图4a4b展示了不同参数组合下的训练与验证损失曲线。整体而言，训练初期损失下降迅速，后期逐渐趋于平稳。

最后，我给出交互模块 `QA_interaction.py` 的一个测试样例。

```

93 if __name__ == "__main__":
94     context = ("数学与应用数学(强基计划)专业依托数学与统计学院，学院拥有悠久的办学历史、良好的学科声誉，以及业界共识的应用数学与交叉学科领先地位。其前身是创
95     question = "谁教机器学习?"
96
97     answer = answer_question(context, question)
98     print(f"问题: {question}")
99     print(f"答案: {answer}")
100
101 # context = "数学与应用数学(强基计划)专业依托数学与统计学院，学院拥有悠久的办学历史、良好的学科声誉，以及业界共识的应用数学与交叉学科领先地位。其前身是创

```

运行 QA_interaction

```

C:\Users\17519\.conda\envs\pytorch\python.exe D:\机器学习大报告\第二次\代码\QA_Interaction.py
C:\Users\17519\.conda\envs\pytorch\Lib\site-packages\torch\nn\functional.py:260: UserWarning: torch.nn.functional.scaled_dot_product_attention()
attn_output = torch.nn.functional.scaled_dot_product_attention(
问题: 谁教机器学习?
答案: 孟德宇老师
进程已结束，退出代码为 0

```

图 5: 自编测试样例

context = 数学与应用数学（强基计划）专业依托数学与统计学院，学院拥有悠久的办学历史、良好的学科声誉，以及业界共识的应用数学与交叉学科领先地位。其前身是创建于 1928 年的交通大学数学系，其数学学科连续两次教育部学科评估均在前 5%（A 档），ESI 科排名位列全球学前 1%。强基计划数学类的课程设置非常丰富——机器学习就是西安交通大学数学强基专业大三下学期的一门专业课，任课教师是数学学院的孟德宇老师。

question = 谁教机器学习？

输出：孟德宇老师

3.2 问题与解决措施

刚开始，我使用原版的 `train-v1.1.json` 作为训练集。代码中加入的进度条模块提示我，完整的一次训练需要 1.5h 左右（如图6）。我只能作罢，取 `train-v1.1.json` 的前一半作为训练集，成功将训练时间缩短到 50min 以内。

```
ome weights of BertForQuestionAnswering were not initialized from the model checkpoint at l
ou should probably TRAIN this model on a down-stream task to be able to use it for predicti
poch 1/1 Training:  0%|          | 0/13882 [00:00<?, ?it/s]C:\Users\17519\conda\envs\pyto
  attn_output = torch.nn.functional.scaled_dot_product_attention(
poch 1/1 Training:  0%|          | 31/13882 [00:13<1:34:14,  2.45it/s, Train Loss=5.15]for
mage          PC          Routine          Line          Source
ibifcoremd.dll 00007FFF6D87DF54 Unknown          Unknown Unknown
ERNELBASE.dll  00007FF99E1BFE4D Unknown          Unknown Unknown
ERNEL32.DLL    00007FF99F18E8D7 Unknown          Unknown Unknown
tdll.dll       00007FF9A075BF6C Unknown          Unknown Unknown
主程已结束，退出代码为 0
```

图 6: 将 `train-v1.1.json` 作为训练集

事实上，我在 `QA_test.py` 上第一次测试模型时，其在 `test` 数据集上的回答正确率仅 5%（忘记截图了，也没保存第一版的 `QA_test.py` 的代码）。因为我实现尝试过几个自己编的个例，发现模型都能根据上下文正确回答我的问题（事后回顾发现，这些例子都是纯中文的，如图5），所以在对模型进行测试之前，我预期模型的正确率应该至少能到 30% 以上的。这样离谱的结果迫使我去寻找原因（模型最终正确率仅在三、四成左右，也有可能是因为其他各种格式问题导致的）。

```
C:\Users\17519\.conda\envs\pytorch\Lib\site-pac
    attn_output = torch.nn.functional.scaled_dot_
文本：美国广播公司（ABC）是一家美国商业广播电视网络公司，
问题：什么公司拥有美国广播公司？
答案：迪 斯 尼 - [UNK] 电 视 集 团

进程已结束，退出代码为 0
```

图 7: 在混合语言回答任务中出现 [UNK]

我在测试集中找了两条记录，放入 QA_interaction.py 中进行测试，结果真的发现了问题。例如图7，输出的结果出现了 [UNK] 字样¹。

在网上搜索以后，我的解决方案是，如果有未知字符，确定答案的起止 token 位，使用直接从原文中进行复制输出为答案。使用了这种方法后，模型在测试集上的正确率从 5% 提升到了 36%（如图3c）。

参考文献

- [1] 苏剑林. 【语料】百度的中文问答数据集 webqa, Apr 2017.
- [2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019.

¹输入的完整文段如下：美国广播公司（ABC）是一家美国商业广播电视网络公司，隶属于迪斯尼-ABC电视集团，是迪斯尼公司迪斯尼媒体网络分部的子公司。这个电视网是三大电视网的一部分。该网络的总部设在哥伦布大道和曼哈顿西 66 街，在纽约市、洛杉矶和加州伯班克设有其他主要办事处和生产设施。

A 附录：实验代码

A.1 QA_model.py

基本参数设置：

```
1 import torch
2 from torch.utils.data import Dataset, DataLoader
3 from transformers import BertTokenizerFast, BertForQuestionAnswering
4 from torch.optim import AdamW
5 import json
6 from tqdm import tqdm
7 import matplotlib.pyplot as plt
8 import seaborn as sns
9
10 # 设置绘图风格，使用seaborn的美观配色
11 sns.set_style("whitegrid")
12 sns.set_palette("husl")
13
14 # 配置参数
15 MAX_LEN = 512
16 BATCH_SIZE = 4 # 减小batch_size防止显存不足
17 EPOCHS = 1
18 LEARNING_RATE = 3e-5
19 TRAIN_PATH = "./ChineseSquad/train.json"
20 DEV_PATH = "./ChineseSquad/dev.json"
21 MODEL_PATH = "./bert_qa_model_512_3e-5_4_128"
22 STRIDE = 128 # 滑动窗口的步长
```

数据预处理模块：

```
1 def load_data(file_path):
2     with open(file_path, 'r', encoding='utf-8') as f:
3         data = json.load(f)
4         return data['data']
5
6
7 def preprocess_data(data):
8     processed = []
9     for article in data:
```

```

10     for paragraph in article['paragraphs']:
11         context = paragraph['context']
12         for qa in paragraph['qas']:
13             # 增强空答案处理
14             if not qa.get('answers') or len(qa['answers']) == 0:
15                 continue
16             answer = qa['answers'][0]
17             # 验证answer_start有效性
18             if answer['answer_start'] >= len(context):
19                 continue
20             processed.append({
21                 'context': context,
22                 'question': qa['question'],
23                 'answer_text': answer['text'],
24                 'start_pos': answer['answer_start'],
25                 'end_pos': answer['answer_start'] + len(answer['text'])
26             })
27     return processed

```

构建数据集类:

```

1 class SquadDataset(Dataset):
2     def __init__(self, data, tokenizer):
3         self.data = data
4         self.tokenizer = tokenizer
5
6     def __len__(self):
7         return len(self.data)
8
9     def __getitem__(self, idx):
10         item = self.data[idx]
11         encodings = self.tokenizer.encode_plus(
12             item['question'],
13             item['context'],
14             add_special_tokens=True,
15             max_length=MAX_LEN,
16             padding='max_length',
17             truncation=True,
18             return_offsets_mapping=True,

```

```

19         stride=STRIDE,
20         return_overflowing_tokens=True
21     )
22
23     input_ids = encodings['input_ids']
24     attention_mask = encodings['attention_mask']
25     offset_mapping = encodings['offset_mapping']
26     overflow_to_sample_mapping =
27         encodings['overflow_to_sample_mapping']
28
29     start_pos = item['start_pos']
30     end_pos = item['end_pos']
31
32     all_start_positions = []
33     all_end_positions = []
34
35     for i in range(len(input_ids)):
36         sample_idx = overflow_to_sample_mapping[i]
37         offset = offset_mapping[i]
38         start_idx = end_idx = 0
39
40         for token_idx, (char_start, char_end) in enumerate(offset):
41             if char_start <= start_pos < char_end:
42                 start_idx = token_idx
43             if char_start < end_pos <= char_end:
44                 end_idx = token_idx
45             if char_end > end_pos:
46                 break
47
48         all_start_positions.append(start_idx)
49         all_end_positions.append(end_idx)
50
51     result = []
52     for i in range(len(input_ids)):
53         result.append({
54             'input_ids': torch.tensor(input_ids[i], dtype=torch.long),
55             'attention_mask': torch.tensor(attention_mask[i],
56                 dtype=torch.long),
57             'start_positions': torch.tensor(all_start_positions[i],

```

```

dtype=torch.long),
56     'end_positions': torch.tensor(all_end_positions[i],
dtype=torch.long)
57     })
58
59     return result

```

训练函数:

```

1 def train():
2     # 初始化模型和FastTokenizer
3     tokenizer = BertTokenizerFast.from_pretrained('bert-base-chinese')
4     model = BertForQuestionAnswering.from_pretrained('bert-base-chinese')
5     device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
6     model.to(device)
7
8     # 加载数据
9     train_data = preprocess_data(load_data(TRAIN_PATH))
10    dev_data = preprocess_data(load_data(DEV_PATH))
11
12    train_dataset = SquadDataset(train_data, tokenizer)
13    dev_dataset = SquadDataset(dev_data, tokenizer)
14
15    # 展开数据集
16    train_flat_dataset = []
17    for item in train_dataset:
18        train_flat_dataset.extend(item)
19    dev_flat_dataset = []
20    for item in dev_dataset:
21        dev_flat_dataset.extend(item)
22
23    train_loader = DataLoader(train_flat_dataset, batch_size=BATCH_SIZE,
shuffle=True)
24    dev_loader = DataLoader(dev_flat_dataset, batch_size=BATCH_SIZE)
25
26    # 使用PyTorch的AdamW优化器
27    optimizer = AdamW(model.parameters(), lr=LEARNING_RATE)
28
29    train_losses = []

```

```

30     val_losses = []
31     step_count = 0
32
33     # 训练循环
34     for epoch in range(EPOCHS):
35         model.train()
36         total_loss = 0
37
38         # 使用tqdm显示训练进度
39         train_progress = tqdm(train_loader, desc=f'Epoch {epoch +
40                                     1}/{EPOCHS} Training')
41         for batch in train_progress:
42             optimizer.zero_grad()
43
44             inputs = {
45                 'input_ids': batch['input_ids'].to(device),
46                 'attention_mask': batch['attention_mask'].to(device),
47                 'start_positions': batch['start_positions'].to(device),
48                 'end_positions': batch['end_positions'].to(device)
49             }
50
51             outputs = model(**inputs)
52             loss = outputs.loss
53             loss.backward()
54             optimizer.step()
55
56             total_loss += loss.item()
57             train_losses.append(loss.item())
58             step_count += 1
59             train_progress.set_postfix({'Train Loss': total_loss /
60                                     (train_progress.n + 1)})
61
62         # 验证损失
63         model.eval()
64         val_loss = 0
65         # 使用tqdm显示验证进度
66         dev_progress = tqdm(dev_loader, desc=f'Epoch {epoch +
67                                     1}/{EPOCHS} Validation')
68         with torch.no_grad():

```

```

66         for batch in dev_progress:
67             inputs = {
68                 'input_ids': batch['input_ids'].to(device),
69                 'attention_mask': batch['attention_mask'].to(device),
70                 'start_positions': batch['start_positions'].to(device),
71                 'end_positions': batch['end_positions'].to(device)
72             }
73             outputs = model(**inputs)
74             val_loss += outputs.loss.item()
75             val_losses.append(outputs.loss.item())
76             dev_progress.set_postfix({'Val Loss': val_loss /
77                                     (dev_progress.n + 1)})
78
79 # 绘制训练损失和验证损失曲线，设置颜色
80 plt.figure(figsize=(10, 6))
81 plt.plot(range(len(train_losses)), train_losses, label='Training
82          Loss', marker='o', color='#70FFE2')
83 plt.plot(range(len(val_losses)), val_losses, label='Validation
84          Loss', marker='s', color='#FF6C6C')
85 plt.title('Training and Validation Loss by Step')
86 plt.xlabel('Training Step')
87 plt.ylabel('Loss')
88 plt.legend()
89 plt.grid(True)
90 plt.tight_layout()
91 plt.savefig(f"{MODEL_PATH}.png")
92 plt.show()
93
94 # 保存模型
95 model.save_pretrained(MODEL_PATH)
96 tokenizer.save_pretrained(MODEL_PATH)
97 return model, tokenizer

```

预测函数:

```

1 def answer_question(context, question, model, tokenizer):
2     device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
3
4     inputs = tokenizer.encode_plus(

```

```

5         question,
6         context,
7         add_special_tokens=True,
8         max_length=MAX_LEN,
9         padding='max_length',
10        truncation=True,
11        return_tensors="pt"
12    ).to(device)
13
14    with torch.no_grad():
15        outputs = model(**inputs)
16
17        start_idx = torch.argmax(outputs.start_logits)
18        end_idx = torch.argmax(outputs.end_logits) + 1
19
20        tokens = tokenizer.convert_ids_to_tokens(inputs["input_ids"][0])
21        answer_tokens = tokens[start_idx:end_idx]
22        answer = tokenizer.convert_tokens_to_string(answer_tokens)
23
24        answer = answer.replace('[CLS]', '').replace('[SEP]', '').strip()
25    return answer

```

主函数:

```

1 if __name__ == "__main__":
2     # 训练模型
3     model, tokenizer = train()
4     print("训练完毕。")

```

A.2 QA_test.py

```

1 import torch
2 from transformers import AutoTokenizer, AutoModelForQuestionAnswering
3 import json
4 from tqdm import tqdm
5
6 # 加载数据、模型、分词器
7 device = "cuda" if torch.cuda.is_available() else "cpu"

```

```

8 data_path = './ChineseSquad/test.json'
9 model_path = './bert_qa_model_512_3e-5_4_128'
10
11 tokenizer = AutoTokenizer.from_pretrained(model_path)
12 model =
13     AutoModelForQuestionAnswering.from_pretrained(model_path).to(device)
14
15 with open(data_path, 'r', encoding='utf-8') as f:
16     data = json.load(f)
17
18 total_questions = 0
19 correct_answers = 0
20 max_question_len = 60
21 max_paragraph_len = 150
22 doc_stride = 150
23 max_seq_len = 1 + max_question_len + 1 + max_paragraph_len + 1
24
25 # 设置种子
26 def same_seeds(seed):
27     torch.manual_seed(seed)
28     if torch.cuda.is_available():
29         torch.cuda.manual_seed(seed)
30         torch.cuda.manual_seed_all(seed)
31     torch.backends.cudnn.benchmark = False
32     torch.backends.cudnn.deterministic = True
33
34 same_seeds(42)

```

构建预测函数：

```

1 def padding(ids_q, ids_p):
2     pad_len = max_seq_len - len(ids_q) - len(ids_p)
3     input_ids = ids_q + ids_p + [0] * pad_len
4     token_type_ids = [0] * len(ids_q) + [1] * len(ids_p) + [0] * pad_len
5     attention_mask = [1] * (len(ids_q) + len(ids_p)) + [0] * pad_len
6     return input_ids, token_type_ids, attention_mask
7
8
9 def evaluate(context, input_ids_list, offset_mapping_list, outputs):

```

```

10 max_prob = -float('inf')
11 best_ans = ''
12 for k in range(len(input_ids_list)):
13     start_logits = outputs.start_logits[k]
14     end_logits = outputs.end_logits[k]
15     start_prob, start_idx = torch.max(start_logits, dim=0)
16     end_prob, end_idx = torch.max(end_logits, dim=0)
17     start_idx = start_idx.item()
18     end_idx = end_idx.item()
19     # 添加边界检查
20     offset_mapping = offset_mapping_list[k]
21     if start_idx < len(offset_mapping) and end_idx <
        len(offset_mapping) and start_idx <= end_idx:
22         if start_prob + end_prob > max_prob:
23             max_prob = start_prob + end_prob
24             # 通过offset_mapping定位原始文本
25             start_char = offset_mapping[start_idx][0]
26             end_char = offset_mapping[end_idx][1]
27             best_ans = context[start_char:end_char].strip()
28 return best_ans

```

回答模块:

```

1 def answer_question(context, question):
2     # 分词并获取offset_mapping
3     tokenized_question = tokenizer(question, add_special_tokens=False)
4     tokenized_paragraph = tokenizer(context, add_special_tokens=False,
        return_offsets_mapping=True)
5
6     input_ids_list = []
7     token_type_ids_list = []
8     attention_mask_list = []
9     offset_mapping_list = [] # 新增: 保存每个分块的offset_mapping
10
11 for i in range(0, len(tokenized_paragraph["input_ids"]), doc_stride):
12     # 处理问题部分
13     input_ids_q = [101] +
        tokenized_question["input_ids"][:max_question_len] + [102]
14     # 处理段落分块

```

```

15     chunk_ids = tokenized_paragraph["input_ids"][i:i +
16         max_paragraph_len]
17     chunk_offset = tokenized_paragraph["offset_mapping"][i:i +
18         max_paragraph_len]
19     input_ids_p = chunk_ids + [102]
20     input_ids, token_type_ids, attention_mask = padding(input_ids_q,
21         input_ids_p)
22
23     input_ids_list.append(input_ids)
24     token_type_ids_list.append(token_type_ids)
25     attention_mask_list.append(attention_mask)
26     # 将offset_mapping对齐到分块后的input_ids_p
27     offset_mapping = [(0, 0)] * len(input_ids_q) + chunk_offset +
28         [(0, 0)] # [CLS]Q[SEP]P[SEP]
29     offset_mapping_list.append(offset_mapping)
30
31     input_ids = torch.tensor(input_ids_list).to(device)
32     token_type_ids = torch.tensor(token_type_ids_list).to(device)
33     attention_mask = torch.tensor(attention_mask_list).to(device)
34
35     with torch.no_grad():
36         outputs = model(
37             input_ids=input_ids,
38             token_type_ids=token_type_ids,
39             attention_mask=attention_mask
40         )
41
42     return evaluate(context, input_ids_list, offset_mapping_list,
43         outputs)

```

统计正确率:

```

1 # 统计问题总数
2 total_questions = sum(len(paragraph['qas']) for item in data['data']
3     for paragraph in item['paragraphs'])
4
5 # 遍历数据，以问题总数为总任务设置进度条
6 with tqdm(total=total_questions, desc="Processing questions") as pbar:
7     for item in data['data']:

```

```

7     for paragraph in item['paragraphs']:
8         context = paragraph['context']
9         for qa in paragraph['qas']:
10             question = qa['question']
11             # 假设我们已经知道标准答案
12             standard_answers = [ans['text'] for ans in qa['answers']]
13             # 这里模拟你输入的答案
14             # 我们直接用标准答案作为输入答案来测试
15             input_answer = answer_question(context, question)
16             if input_answer in standard_answers:
17                 correct_answers += 1
18             pbar.update(1)
19
20 # 计算正确率
21 accuracy = correct_answers / total_questions if total_questions > 0
22         else 0
23
24 print(f"使用模型: {model_path}")
25 print(f"总问题个数: {total_questions}")
26 print(f"回答正确问题个数: {correct_answers}")
27 print(f"正确率: {accuracy:.2%}")

```

A.3 QA_Interaction.py

基本参数设置:

```

1 import torch
2 from transformers import AutoTokenizer, AutoModelForQuestionAnswering
3
4 def same_seeds(seed):
5     torch.manual_seed(seed)
6     if torch.cuda.is_available():
7         torch.cuda.manual_seed(seed)
8         torch.cuda.manual_seed_all(seed)
9     torch.backends.cudnn.benchmark = False
10    torch.backends.cudnn.deterministic = True
11
12

```

```
13 device = "cuda" if torch.cuda.is_available() else "cpu"
14 same_seeds(42)
```

加载模型:

```
1 # 加载模型和分词器
2 tokenizer =
    AutoTokenizer.from_pretrained("./bert_qa_model_512_3e-5_4_128")
3 model =
    AutoModelForQuestionAnswering.from_pretrained("./bert_qa_model_512_3e-5_4_128").to(device)
4 model.eval()
5
6 max_question_len = 60
7 max_paragraph_len = 150
8 doc_stride = 150
9 max_seq_len = 1 + max_question_len + 1 + max_paragraph_len + 1
10
11
12 def padding(ids_q, ids_p):
13     pad_len = max_seq_len - len(ids_q) - len(ids_p)
14     input_ids = ids_q + ids_p + [0] * pad_len
15     token_type_ids = [0] * len(ids_q) + [1] * len(ids_p) + [0] * pad_len
16     attention_mask = [1] * (len(ids_q) + len(ids_p)) + [0] * pad_len
17     return input_ids, token_type_ids, attention_mask
18
19
20 def evaluate(context, input_ids_list, offset_mapping_list, outputs):
21     max_prob = -float('inf')
22     best_ans = ''
23     for k in range(len(input_ids_list)):
24         start_logits = outputs.start_logits[k]
25         end_logits = outputs.end_logits[k]
26         start_prob, start_idx = torch.max(start_logits, dim=0)
27         end_prob, end_idx = torch.max(end_logits, dim=0)
28         if start_prob + end_prob > max_prob and start_idx <= end_idx:
29             max_prob = start_prob + end_prob
30             # 通过offset_mapping定位原始文本
31             offset_mapping = offset_mapping_list[k]
32             start_char = offset_mapping[start_idx][0]
```

```

33         end_char = offset_mapping[end_idx][1]
34         best_ans = context[start_char:end_char].strip()
35     return best_ans

```

回答模块:

```

1 def answer_question(context, question):
2     # 分词并获取offset_mapping
3     tokenized_question = tokenizer(question, add_special_tokens=False)
4     tokenized_paragraph = tokenizer(context, add_special_tokens=False,
5                                     return_offsets_mapping=True)
6
7     input_ids_list = []
8     token_type_ids_list = []
9     attention_mask_list = []
10    offset_mapping_list = [] # 新增: 保存每个分块的offset_mapping
11
12    for i in range(0, len(tokenized_paragraph["input_ids"]), doc_stride):
13        # 处理问题部分
14        input_ids_q = [101] +
15                      tokenized_question["input_ids"][:max_question_len] + [102]
16        # 处理段落分块
17        chunk_ids = tokenized_paragraph["input_ids"][i:i +
18              max_paragraph_len]
19        chunk_offset = tokenized_paragraph["offset_mapping"][i:i +
20              max_paragraph_len]
21        input_ids_p = chunk_ids + [102]
22        input_ids, token_type_ids, attention_mask = padding(input_ids_q,
23              input_ids_p)
24
25        input_ids_list.append(input_ids)
26        token_type_ids_list.append(token_type_ids)
27        attention_mask_list.append(attention_mask)
28        # 将offset_mapping对齐到分块后的input_ids_p
29        offset_mapping = [(0, 0)] * len(input_ids_q) + chunk_offset +
30              [(0, 0)] # [CLS]Q[SEP]P[SEP]
31        offset_mapping_list.append(offset_mapping)
32
33    input_ids = torch.tensor(input_ids_list).to(device)

```

```

28 token_type_ids = torch.tensor(token_type_ids_list).to(device)
29 attention_mask = torch.tensor(attention_mask_list).to(device)
30
31 with torch.no_grad():
32     outputs = model(
33         input_ids=input_ids,
34         token_type_ids=token_type_ids,
35         attention_mask=attention_mask
36     )
37
38 return evaluate(context, input_ids_list, offset_mapping_list,
                 outputs)

```

使用主函数进行交互：

```

1 if __name__ == "__main__":
2     context =
3         "美国广播公司（ABC）是一家美国商业广播电视网络公司，隶属于..."
4     question = "什么公司拥有美国广播公司？"
5
6     answer = answer_question(context, question)
7     print(f"问题: {question}")
8     print(f"答案: {answer}")
9
10    # context =
11        "数学与应用数学（强基计划）专业依托数学与统计学院，学院拥有悠久..."
12    # question = "谁教机器学习？"

```